

南開大學
Nankai University

计算机学院

《人工智能导论》课程实验报告

垃圾短信分类

姓名：赵宇哲

学号：2411094

专业：计算机科学与技术

实验时间：2025 年 11 月 10 日 星期一

目录

1 问题重述 **2**

2 设计思想 **3**

2.1 模型构建与训练 3

2.2 利用训练模型进行预测 4

3 程序代码 **4**

4 测试结果 **8**

4.1 模型训练结果 8

4.2 在线测试结果 9

5 总结 **9**

1 问题重述

实验背景

垃圾短信 (Spam Messages，SM) 是指未经过用户同意向用户发送不愿接收的商业广告或者不符合法律规范的短信。随着手机的普及，垃圾短信在日常生活日益泛滥，已经严重的影响到了人们的正常生活娱乐，乃至社会的稳定。据 360 公司 2020 年第一季度有关手机安全的报告提到，360 手机卫士在第一季度共拦截各类垃圾短信约 34.4 亿条，平均每日拦截垃圾短信约 3784.7 万条。大数据时代的到来使得大量个人信息数据得以沉淀和积累，但是庞大的数据量缺乏有效的整理规范；在面对量级如此巨大的短信数据时，为了保证更良好的用户体验，如何从数据中挖掘出更多有意义的信息为人们免受垃圾短信骚扰成为当前亟待解决的问题。

实验要求

任务提供包括数据读取、基础模型、模型训练等基本代码。参赛选手需完成核心模型构建代码，并尽可能将模型调到最佳状态。模型单次推理时间不超过 10 秒。

实验环境

可以使用基于 Python 的 Pandas、Numpy、Sklearn 等库进行相关特征处理，使用 Sklearn 框架训练分类器，也可编写深度学习模型，使用过程中请注意 Python 包（库）的版本。

注意事项

Python 与 Python Package 的使用方式，可在右侧 API 文档中查阅。当右上角的『Python 3』长时间指示为运行中的时候，造成代码无法执行时，可以重新启动 Kernel 解决（左上角『Kernel』-『Restart Kernel』）。

2 设计思想

2.1 模型构建与训练

首先，我们读取数据集并进行**欠采样（Undersampling）**处理。为了解决正负样本比例失衡的问题（正常短信远多于垃圾短信），我们将负样本的数量限制为与正样本一致，构建 1:1 的平衡数据集。

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import MaxAbsScaler
from sklearn.naive_bayes import MultinomialNB
import joblib
import os

data_path = "./datasets/5f9ae242cae5285cd734b91e-momodel/sms_pub.csv"
sms = pd.read_csv(data_path, encoding='utf-8')
sms.dropna(inplace=True)

# 执行欠采样，确保正负样本 1:1
sms_pos = sms[(sms['label'] == 1)]
n_neg_samples = min(len(sms_pos), len(sms[(sms['label'] == 0)]))
sms_neg = sms[(sms['label'] == 0)].sample(n=n_neg_samples, random_state=42)
sms = pd.concat([sms_pos, sms_neg], axis=0).sample(frac=1.0, random_state=42)
```

Listing 1: 数据预处理

然后，我们读取停用词库，用于后续的文本向量化过滤。

```
stopwords_path = r'scu_stopwords.txt'
def read_stopwords(stopwords_path):
    try:
        with open(stopwords_path, 'r', encoding='utf-8') as f:
            stopwords = f.read().splitlines()
    except FileNotFoundError:
        stopwords = []
    return stopwords
stopwords = read_stopwords(stopwords_path)
```

Listing 2: 读取停用词库

接着，我们划分训练集与测试集，并构建 Pipeline。这里我们利用 `TfidfVectorizer`（开启 1−2 gram 上下文特征、限制最大特征数 50000）、`MaxAbsScaler`（稀疏矩阵归一化）和 `MultinomialNB`（多项式朴素贝叶斯）构建流水线。模型在训练集上拟合后，在测试集上验证效果。

```
X = np.array(sms.msg_new)
y = np.array(sms.label)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=22, test_size=0.2)

pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(
        stop_words=stopwords,
        ngram_range=(1,2),      # 引入二元语法特征
        max_features=50000,     # 限制特征维度
        sublinear_tf=True,      # 亚线性平滑
```

```
        token_pattern=r"(?u)\b\w+\b"
    )),
    ('MaxAbsScaler', MaxAbsScaler()),  # 归一化处理
    ('classifier', MultinomialNB(alpha=0.01)), # 设置平滑参数 alpha=0.01
])

pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)
```

Listing 3: 构建 Pipeline

最后，为了提高模型的泛化能力，我们在全量数据集（X, y）上重新训练模型，并将最终模型保存到指定路径。

```
pipeline.fit(X, y) # 全量重训
os.makedirs('results', exist_ok=True)
pipeline_path = 'results/pipeline_now_the_best.model'
joblib.dump(pipeline, pipeline_path, compress=3)
```

Listing 4: 全量重训并保存模型

2.2 利用训练模型进行预测

首先，我们加载保存好的模型文件 `pipeline_now_the_best.model`。由于 Pipeline 封装了数据处理流程，预测时无需再次手动处理停用词或向量化。

```
import joblib
pipeline_path = 'results/pipeline_now_the_best.model'
pipeline = joblib.load(pipeline_path)
```

Listing 5: 加载模型

然后，定义 `predict` 函数。输入分词后的字符串，直接利用 pipeline 进行预测，返回预测的 *label*（类别）和 *proba*（概率分布）。

```
def predict(message):
    label = int(pipeline.predict([message])[0])
    proba = list(pipeline.predict_proba([message])[0])
    return label, proba
```

Listing 6: 预测函数

3 程序代码

```
import os
import warnings
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import MaxAbsScaler
from sklearn.naive_bayes import MultinomialNB
```

```
from sklearn.metrics import (
    roc_auc_score,
    f1_score,
    classification_report,
    confusion_matrix,
    accuracy_score
)

# 1. 基础配置与兼容性
os.environ["HDF5_USE_FILE_LOCKING"] = "FALSE"
warnings.filterwarnings('ignore')

try:
    import joblib
except ImportError:
    from sklearn.externals import joblib

# 确保数据路径正确
data_path = "./datasets/5f9ae242cae5285cd734b91e-momodel/sms_pub.csv"
stopwords_path = r'scu_stopwords.txt'

def read_stopwords(stopwords_path):
    stopwords = []
    try:
        with open(stopwords_path, 'r', encoding='utf-8') as f:
            stopwords = f.read().splitlines()
    except FileNotFoundError:
        print(f"[WARN] 停用词文件 {stopwords_path} 未找到，将不使用停用词。")
        pass
    return stopwords

if __name__ == "__main__":
    # 2. 读取数据
    if not os.path.exists(data_path):
        print(f"[ERROR] 数据文件未找到: {data_path}")

    print("[INFO] 读取数据...")
    sms = pd.read_csv(data_path, encoding='utf-8')
    sms = sms.dropna(subset=['msg_new', 'label'])

    # ============================================================
    # 欠采样 (Undersampling)
    # ============================================================
    print("[INFO] 执行欠采样 (1:1 平衡)...")
    sms_pos = sms[(sms['label'] == 1)]
    n_neg_samples = min(len(sms_pos), len(sms[(sms['label'] == 0)]))
    sms_neg = sms[(sms['label'] == 0)].sample(n=n_neg_samples, random_state=42)

    sms = pd.concat([sms_pos, sms_neg], axis=0).sample(frac=1.0, random_state=42)

    stopwords = read_stopwords(stopwords_path)

    X = np.array(sms.msg_new)
    y = np.array(sms.label)

    X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=22, test_size=0.2)

    print(f"Total samples used: {X.shape[0]}")
    print(f"Train samples: {X_train.shape[0]}")
```

```
    print(f"Test samples: {X_test.shape[0]}")

    # 3. 构建 Pipeline
    pipeline = Pipeline([
        ('tfidf', TfidfVectorizer(
            stop_words=stopwords or None,
            ngram_range=(1,2),
            max_features=50000,
            sublinear_tf=True,
            token_pattern=r"(?u)\b\w+\b"
        )),
        ('MaxAbsScaler', MaxAbsScaler()),
        ('classifier', MultinomialNB(alpha=0.01)),
    ])

    # 4. 训练与评估 (已补全所有指标)
    print("-" * 30)
    print("[INFO] 开始训练 (验证集)...")
    pipeline.fit(X_train, y_train)
    y_pred = pipeline.predict(X_test)

    # --- 输出指标 ---
    # 准确率
    acc = accuracy_score(y_test, y_pred)
    print(f"\n[验证集] 准确率 (Accuracy): {acc:.4f}")

    # F1 Score
    f1 = f1_score(y_test, y_pred)
    print(f"[验证集] F1-score: {f1:.4f}")

    # AUC Score
    try:
        y_proba = pipeline.predict_proba(X_test)[:, 1]
        auc = roc_auc_score(y_test, y_proba)
        print(f"[验证集] AUC Score: {auc:.4f}")
    except:
        pass

    # 混淆矩阵
    print("\n[验证集] 混淆矩阵 (Confusion Matrix):")
    print(confusion_matrix(y_test, y_pred))

    # 分类报告
    print("\n[验证集] 详细分类报告 (Classification Report):")
    print(classification_report(y_test, y_pred))
    print("-" * 30)

    # 5. 全量重训并保存
    print("[INFO] 全量重训 (使用所有数据)...")
    pipeline.fit(X, y)

    os.makedirs('results', exist_ok=True)

    pipeline_path = 'results/pipeline_now_the_best.model'

    print(f"[INFO] 正在保存模型到 {pipeline_path} ...")
    joblib.dump(pipeline, pipeline_path, compress=3)

    file_size = os.path.getsize(pipeline_path) / 1024
```

```
    print(f"[SUCCESS] 模型已保存。文件大小: {file_size:.2f} KB")
```

Listing 7: train.py

```
import os
os.environ["HDF5_USE_FILE_LOCKING"] = "FALSE"
stopwords_path = r'scu_stopwords.txt'

def read_stopwords(stopwords_path):

    stopwords = []
    try:
        with open(stopwords_path, 'r', encoding='utf-8') as f:
            stopwords = f.read().splitlines()
    except FileNotFoundError:
        print(f"错误: 停用词文件 {stopwords_path} 未找到。")
    return stopwords

stopwords = read_stopwords(stopwords_path)

from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.naive_bayes import BernoulliNB
from sklearn.naive_bayes import MultinomialNB
from sklearn.naive_bayes import ComplementNB
from sklearn.preprocessing import MaxAbsScaler

pipeline_list = [

    # 步骤1: 向量化
    ('vectorizer', TfidfVectorizer(
        stop_words=stopwords,
        ngram_range=(1, 1),
        max_features=50000
    )),

    # 步骤2: 数据缩放
    ('scaler', MaxAbsScaler()),

    # 步骤3: 分类器
    # 保持 10/10 的设置: alpha=1.0
    ('classifier', ComplementNB(alpha=1.0))

]

from sklearn.externals import joblib
pipeline_path = 'results/pipeline_now_the_best.model'
# ------------------------------------------------
pipeline = joblib.load(pipeline_path)

def predict(message):
    label = pipeline.predict([message])[0]
    proba = list(pipeline.predict_proba([message])[0])

    return label, proba
```

Listing 8: main.py

4 测试结果

4.1 模型训练结果

```
[验证集] 准确率 (Accuracy): 0.9734
[验证集] F1-score: 0.9739
[验证集] AUC Score: 0.9966

[验证集] 混淆矩阵 (Confusion Matrix):
[[15113   653]
 [  188 15705]]

[验证集] 详细分类报告 (Classification Report):
              precision    recall  f1-score   support

           0       0.99      0.96      0.97     15766
           1       0.96      0.99      0.97     15893

    accuracy                           0.97     31659
   macro avg       0.97      0.97      0.97     31659
weighted avg       0.97      0.97      0.97     31659
```

图 4.1.1: 模型训练结果

F1-score：0.9739。作为精确率和召回率的调和平均数，该高分证明模型在正负样本上达到了良好的平衡。

AUC Score：0.9966。接近 1.0 的 AUC 值表明模型具有极强的排序能力，能够非常自信地区分垃圾短信与正常短信。

4.2 在线测试结果



图 4.2.1: 模型训练结果

我们在网站中进行线上测试，测试用时给 5s（10s 以内），且得到了 10/10 的结果。

5 总结

本实验通过编写 train.py 脚本，采用朴素贝叶斯分类器和 TF-IDF 特征提取方法完成了模型的训练，并将训练好的最佳模型保存至 results 文件夹下；随后利用 main.py 调用该模型实现了对垃圾短信的预测功能。根据实验结果，该方法在测试集上的 F1-score 和 AUC 上表现良好，达到了预期目标。

当然，目前的欠采样策略虽然有效，但也损失了部分正常短信的数据信息。未来我们可以尝试过采样（如 SMOTE）或集成学习等方法来实现模型的优化，也可以选择其他的深度学习算法模型来提升与优化我们的模型。